Learning Simple Concepts on Strings

Software Specification, Revision 2 — with Background

September 2026

Contents

- **How to read this document**

PART I — BACKGROUND

- **1. What problem is this?**
  - 1.1 The three questions any learner must answer
- **2. The vocabulary of concept learning**
  - 2.1 Instances, concepts, examples
  - 2.2 Hypotheses and the hypothesis space
  - 2.3 Consistency
  - 2.4 Generalisation
  - 2.5 Inductive bias
  - 2.6 Occam's razor and the Minimum Description Length principle
  - 2.7 How much data is enough? Identifiability
- **3. Representing a hypothesis as a program**
  - 3.1 Domain-specific languages
  - 3.2 Types
  - 3.3 Expressions, trees, and DAGs
  - 3.4 Syntax versus semantics
  - 3.5 Totality and the bottom value
  - 3.6 Purity
- **4. Searching for a hypothesis**
  - 4.1 The size of the space
  - 4.2 Bottom-up enumerative synthesis
  - 4.3 Observational equivalence
  - 4.4 Evolutionary search
  - 4.5 Constraint-based synthesis
  - 4.6 Comparison
- **5. Assembling a concept from parts**
  - 5.1 Two layers
  - 5.2 Disjunctive normal form
  - 5.3 Sequential covering
  - 5.4 Decision trees as an alternative

  - 5.5 Pruning
- **6. Formal-language background**
  - 6.1 Alphabets, strings, languages
  - 6.2 Regular languages and finite automata
  - 6.3 Where the target concepts sit
  - 6.4 Grammatical inference
  - 6.5 Why still use an expression language?
  - 6.6 A note on expressive power within the DSL
- **7. How results should be measured**
  - 7.1 Train, validation, and test
  - 7.2 Cross-validation
  - 7.3 Metrics
  - 7.4 Boundary sampling
  - 7.5 Reproducibility
  - 7.6 Baselines and ablations
- **8. Glossary**

PART II — REVISED SPECIFICATION

- **9. Scope and goals**
  - 9.1 Purpose
  - 9.2 Functional requirements
  - 9.3 Non-functional requirements
  - 9.4 Explicit non-goals
- **10. Input format**
  - 10.1 Grammar
  - 10.2 Example
  - 10.3 Rules and edge cases
  - 10.4 Label separation — a mandatory correction
- **11. Type system**
  - 11.1 Numeric representation
- **12. Values and bottom**
- **13. Operator set**
  - 13.1 Structural
  - 13.2 Constants
  - 13.3 String to string
  - 13.4 String to integer
  - 13.5 String to character
  - 13.6 String predicates
  - 13.7 Arithmetic
  - 13.8 Comparison and boolean
  - 13.9 Optional: learned automaton

    - 13.10 Guarding STR constants against memorisation
- **14. Hypothesis representation**
    - 14.1 Structure
    - 14.2 Invariants
    - 14.3 Size
- **15. Semantics**
    - 15.1 Definition
    - 15.2 Evaluation with sharing
    - 15.3 Behaviour vectors
    - 15.4 Rendering
- **16. The learning algorithm**
    - 16.1 Overview
    - 16.2 Layer 1 — enumeration
    - 16.3 Layer 2 — sequential covering
    - 16.4 Top level and model selection
    - 16.5 Alternative strategy — evolutionary search
    - 16.6 Baseline strategy — RPNI
- **17. Software architecture**
    - 17.1 Layering
    - 17.2 Responsibilities
    - 17.3 Design patterns
    - 17.4 Class diagram
- **18. Configuration**
- **19. Evaluation protocol**
    - 19.1 Data
    - 19.2 Metrics
    - 19.3 Cost
    - 19.4 Identifiability report
    - 19.5 Required comparisons
- **20. Output**
    - 20.1 Formula
    - 20.2 Graph
    - 20.3 JSON
    - 20.4 Report

PART IV — CGL CONSTRUCTION

- **21. The value bank**
    - 21.1 Why the tree stops being the right picture
    - 21.2 Definition
    - 21.3 Invariants
    - 21.4 Batch construction

  - 21.5 Extraction
  - 21.6 Saturation, and what does not saturate
  - 21.7 Behaviour equivalence is finer than coverage equivalence
  - 21.8 Configuration additions
- **22. Incremental ingestion and partition refinement**
  - 22.1 The conflict
  - 22.2 What ingestion must do
  - 22.3 Invariants
  - 22.4 Identifiability comes out of this for free
  - 22.5 When refinement earns its keep
  - 22.6 Relation to Section 16.2
  - 22.7 What stage 3 must do now

PART III — APPENDICES

- **Appendix A. Worked example: Test Case 1024**
  - A.1 The data
  - A.2 Some behaviour vectors
  - A.3 Enumeration by size
  - A.4 The result is not what the data's author intended
  - A.5 What requirement F6 produces instead
  - A.6 Adding the disambiguating example
  - A.7 What this example demonstrates
- **Appendix B. Change log against the original document**
  - B.1 Blocking corrections
  - B.2 Specification defects
  - B.3 Editorial
  - B.4 What was kept
- **Appendix C. Suggested implementation milestones**
- **Appendix D. References**
  - Concept learning and learning theory
  - Rule learning
  - Program synthesis and genetic programming
  - Grammatical inference
  - Semantics

How to read this document

This document replaces the previous *Software Description Document*. It is longer than the original for one deliberate reason: the original assumed familiarity with a body of work in symbolic machine learning and program synthesis that it never named. Part I supplies that

background from first principles. Part II is the actual revised specification. Part III contains a fully worked example, a change log against the original, and references.

Part I is written for a reader who is comfortable with algorithms, discrete mathematics, and data mining, but who has not worked in symbolic AI. Wherever a concept has a familiar analogue in data mining or in the theory of computation, that connection is drawn explicitly. Nothing in Part I is required reading for someone who only wants to implement Part II, but Part II uses its vocabulary throughout.

The three parts are:

- **Part I — Background.** Sections 1 to 8. What kind of problem this is, the vocabulary of concept learning, why a program is a reasonable thing to learn, how such programs are searched for, and how the result is evaluated.
- **Part II — Specification.** Sections 9 to 20. Input format, type system, operator set, the meaning of a learned hypothesis, the learning algorithm, the software architecture, and the evaluation protocol.
- **Part III — Appendices.** A worked example on the test case from the original document, a change log with the reasoning behind each change, suggested implementation milestones, and references.

PART I — BACKGROUND

1. What problem is this?

The system takes a file containing labelled binary strings and must produce a description of the rule that separates the strings labelled + from those labelled `-`. For example, given

```
+ 1001
+ 00001
- 11100
+ 10
- 111
```

it should output something a human can read, such as “the longest run of consecutive 1s is at most 2”.

This is a **classification** problem, which is familiar territory. What makes it unusual is the form of the answer. A decision tree, a support vector machine, or a neural network would answer with a numeric model over some fixed set of features. Here the answer must be a *program* — an executable expression that anyone can read, that computes the class of a string, and that is meant to be the actual rule rather than an approximation of it.

Learning a program from examples is called **inductive programming** or **program synthesis from examples**. It sits in the branch of AI called **symbolic** AI: the objects being manipulated

are discrete structures (expressions, formulas, graphs) with a defined meaning, rather than continuous vectors of numbers.

Three properties distinguish this setting from ordinary classification:

- **The output is discrete and structured.** There is no gradient to descend. A candidate program is either syntactically valid or not; changing one operator changes its behaviour discontinuously. The search must therefore be combinatorial rather than numerical.
- **A perfect fit is expected.** In statistical learning we accept some training error because the data are noisy. Here the labels come from a precise mathematical rule, so a correct hypothesis should classify every training example correctly. This changes what "good" means and changes which algorithms apply.
- **Interpretability is the point.** The value of the output is that a person can read it. A model that scored 99% but could not be inspected would miss the goal entirely.

1.1 The three questions any learner must answer

Every learning system, symbolic or statistical, has to answer three questions. It is useful to keep them separate, because the original document conflated them and most of the problems in it trace back to that.

1. **What is a hypothesis?** What is the space of possible answers, and what does each answer *mean*? For a decision tree this is easy: a tree, and its meaning is "follow the branches to a leaf". Section 3 answers this for our setting.
2. **What makes one hypothesis better than another?** This is the objective function, loss function, or fitness function. Section 2 develops it.
3. **How do we find a good one?** This is the search algorithm. Sections 4 and 5 cover it.

Keeping these three separate is not merely tidy. It is what allows the implementation to be tested: the hypothesis language can be tested without any search, the objective can be tested on hand-written hypotheses, and the search can be swapped for a different strategy without touching either.

2. The vocabulary of concept learning

This section defines the standard terms. They come mostly from Mitchell's formulation of concept learning, which is the classical framing for exactly this problem.

2.1 Instances, concepts, examples

- The **instance space** X is the set of all objects that could be classified. Here $X = \Sigma^*$, the set of all finite strings over an alphabet Σ. For binary strings, $\Sigma = \{0, 1\}$, and X is infinite.
- A **concept** c is a subset of X, or equivalently a function $c : X \to \{\text{true}, \text{false}\}$. "Strings with an odd number of 1s" is a concept. It is the thing we are trying to learn, and we do not get to see it directly.

- An **example** is a pair `(s, ℓ)` where `s ∈ X` and `ℓ = c(s)`. An example with `ℓ = true` is a **positive example**; one with `ℓ = false` is a **negative example**.
- The **training set** `D` is a finite set of examples. In our input files, a test case is exactly a training set.

The **target concept** is the true `c`. The learner never observes it — only a finite sample of its behaviour. Everything difficult about learning follows from that gap.

2.2 Hypotheses and the hypothesis space

A **hypothesis** `h` is the learner's guess at the target concept. It is also a function `X → {true, false}`, but unlike the target it is something we can write down and execute.

The **hypothesis space** `H` is the set of all hypotheses the learner is capable of producing. This is not an incidental implementation detail; it is the single most important design decision in the whole system. Two facts follow immediately:

- If the target concept is not in `H`, the learner cannot possibly get the right answer, no matter how much data or compute it has. This is called an **expressiveness failure**, and it is the most common way a symbolic learner fails silently.
- If `H` is very large, finding the right member of it becomes computationally hard, and — as Section 2.6 explains — the risk of picking a wrong hypothesis that happens to fit the data goes up.

So `H` must be large enough to contain the target and small enough to be searchable. Choosing it is a modelling decision, and in this project it takes the form of choosing the set of operators (Section 13).

2.3 Consistency

A hypothesis `h` is **consistent** with a training set `D` if it classifies every example in `D` correctly:

```
consistent(h, D)  ⇔  for all (s, ℓ) ∈ D :  h(s) = ℓ
```

Consistency is a property relative to a *finite* sample. It is the natural success criterion in a noise-free setting like ours. But it is weak: usually many hypotheses are consistent with the same small training set, and they disagree with each other on strings not in the training set. Deciding among them is the subject of Sections 2.5 to 2.7.

2.4 Generalisation

Generalisation is how well a hypothesis classifies instances it was not trained on. It is the only thing that ultimately matters, and it is the one thing the training data cannot directly measure.

Consider the example file from Section 1. The hypothesis

```
s = "1001"  or  s = "00001"  or  s = "10"
```

is consistent with the training set. It is also useless: it will call every unseen positive string negative. A learner that produces this is a **rote learner** — it has memorised rather than

generalised. Note carefully that *no* measurement made on the training set can detect this problem, because on the training set the rote learner is perfect. This is why the original document's success criterion ("the operation did not throw an exception") cannot work: it admits the rote learner as a perfect solution.

2.5 Inductive bias

If many hypotheses are consistent with the data, how does a learner choose? It must have a preference that comes from somewhere other than the data. That preference is called the **inductive bias**.

Every learner has one, whether or not it is stated. In a decision tree learner using information gain, the bias is "prefer shorter trees with high-information attributes near the root". In a linear model with L2 regularisation, the bias is "prefer small coefficients". A learner with no inductive bias cannot generalise at all — this is the content of the "no free lunch" observation.

In our setting the bias has two components, and both should be written down explicitly rather than left implicit in the code:

1. **The choice of operators.** By providing `MAXRUN` but not, say, "number of positions where the string differs from its reverse", we are declaring which concepts we consider natural. This is the strongest bias in the system.
2. **A preference for small hypotheses.** This is the subject of the next section.

2.6 Occam's razor and the Minimum Description Length principle

The standard preference among consistent hypotheses is for the **smallest** one. This is Occam's razor, and in learning theory it is more than an aesthetic preference — it has a formal justification.

The intuition: there are few short hypotheses and enormously many long ones. If a *short* hypothesis fits `m` examples, that is unlikely to be a coincidence, because there were not many short hypotheses available to get lucky. If a *long* hypothesis fits `m` examples, that is much less impressive, because among the vast number of long hypotheses some were bound to fit by chance. Formally, if `|H|` is the size of the hypothesis space, the probability that some consistent hypothesis has true error greater than `ε` is at most `|H| · e^(−εm)`. Restricting attention to short hypotheses shrinks the effective `|H|` and therefore tightens the bound.

The **Minimum Description Length** (MDL) principle turns this into a usable objective. It says: choose the hypothesis that minimises

```
L(h) + L(D | h)
```

where `L(h)` is the number of bits needed to write down the hypothesis and `L(D | h)` is the number of bits needed to describe the data *given* the hypothesis — in practice, the cost of listing the examples `h` gets wrong. This makes the trade-off between complexity and fit explicit, and it derives the trade-off weight rather than leaving it as an arbitrary tuning constant.

For our purposes a workable concrete form is:

```
cost(h, D) = errors(h, D) · log₂|D|  +  size(h) · log₂|Ops|
```

The first term charges for each misclassified example; the second charges for each node in the hypothesis, at a rate of the bits needed to name an operator. Minimising `cost` prefers a small consistent hypothesis, and prefers a slightly wrong small hypothesis over a perfect enormous one — which is exactly the guard against the rote learner from Section 2.4.

2.7 How much data is enough? Identifiability

A separate and equally important question: given a training set, is there even enough information in it to pin down the answer?

The **VC dimension** of a hypothesis space is the size of the largest set of instances that the space can classify in all possible ways. It measures the expressive richness of `H`. For a finite space, `VC(H) ≤ log₂|H|`. The standard sample-complexity bound says that to be confident (probability `1 − δ`) that a consistent hypothesis has true error below `ε`, it suffices to have

```
m ≥ (1/ε) · (ln|H| + ln(1/δ))
```

examples. The practical reading is blunt: a hypothesis space of even a few thousand candidates needs tens or hundreds of examples before a consistent hypothesis is trustworthy. **A test case with five examples cannot identify anything**, and the software should say so rather than reporting a confident answer.

Identifiability is the concrete version of this. A training set identifies a concept only if no *other* plausible concept is also consistent with it. Appendix A shows that the five-example test case in the original document does not identify its stated concept — at least five different rules of equal size fit the data exactly. This is not a flaw in the algorithm; it is a flaw in the data, and the software should detect and report it.

Active learning is the natural response. If the learner can ask an oracle "is `11011` positive?" — a **membership query** — it can choose questions that split the remaining candidates, and identify the concept from far fewer examples than random sampling would need. Angluin's `L*` algorithm (Section 6.4) is the classical example. Where a ground-truth oracle is available, this should be offered.

3. Representing a hypothesis as a program

We now answer question 1 from Section 1.1: what is a hypothesis, and what does it mean?

3.1 Domain-specific languages

Rather than let a hypothesis be an arbitrary program in a general-purpose language — which would be both unsearchable and undecidable to reason about — we fix a small **domain-specific language** (DSL). A DSL is a restricted set of operations chosen for one problem domain. Ours contains things like "count the occurrences of a character", "take the length", "compare two numbers", "and", "or", "not".

The DSL *is* the hypothesis space. Everything in Section 2.2 about H applies to it directly. Making the DSL richer makes more concepts reachable but makes search harder and generalisation weaker. This trade-off is the central design tension of the project and should be measured, not guessed at.

3.2 Types

A **type** classifies the kind of value an expression produces. Our DSL has four:

Type	Values
`Str`	finite strings over Σ
`Char`	single symbols from Σ
`Int`	arbitrary-precision integers
`Bool`	`true`, `false`

Every operator has a **signature**: the types it consumes and the type it produces. For example:

```
LEN     : Str → Int
COUNT   : Str × Char → Int
LE      : Int × Int → Bool
AND     : Bool × Bool → Bool
```

Types do two jobs here. First, they prevent nonsense expressions such as adding a string to a boolean. Second — and this is the important one for the search — they *prune*. When the search asks "what can I attach here?", the type of the slot answers it. A slot expecting an `Int` can only be filled by an `Int`-producing expression. Types cut the branching factor of the search by a large constant factor for free. This is why the approach is called **strongly typed** synthesis, and why the type system deserves care.

3.3 Expressions, trees, and DAGs

An **expression** is an operator applied to sub-expressions, recursively. The natural picture is a tree:

```
        LE
       /  \
   MAXRUN   2
    /   \
 INPUT  '1'
```

which reads: "the longest run of `'1'` in the input is at most 2".

If a sub-expression is used in more than one place, sharing it turns the tree into a **directed acyclic graph** (DAG). For example, comparing the count of 1s to the count of 0s uses `INPUT` twice; a DAG stores it once with two outgoing edges. The graph must be **acyclic** — a cycle would be a self-referential expression with no value.

Two vocabulary points, since the original document used its own terms:

- What the original called a **connector** is simply an edge from an expression's output to a slot in a parent expression. The rule "you may connect an output to an input of the same type" is precisely the typing rule from Section 3.2.
- The original distinguished a node's **input** from its **arguments**. That distinction is dropped. An operator has a single ordered list of typed parameters; a constant is just an operator with zero parameters. This is simpler, and it removes a contradiction in the original where arguments were sometimes constants and sometimes the outputs of other nodes.

The **size** of an expression is its node count, counting shared nodes once:

```
size(op(e₁, …, e_k)) = 1 + Σ size(eᵢ)     (shared subexpressions
counted once)
```

This is the quantity Occam's razor minimises.

3.4 Syntax versus semantics

The **syntax** of a hypothesis is its shape as a graph. Its **semantics** is the function it denotes. Giving the semantics means defining, for every operator, what value it produces from its arguments, and then defining the value of a whole expression compositionally from its parts. The result is a function

```
⟦G⟧ : Σ* → Bool
```

This is called a **denotational semantics**: each syntactic object *denotes* a mathematical function. Writing it down is not a formality. Without it, the following questions have no answers, and all of them arose from the original document:

- What does a learned graph return on a string it has never seen?
- If a graph has several endpoints, is its verdict their conjunction or disjunction?
- What happens when a sub-computation fails?

The revised design answers all three by construction: a hypothesis has exactly one output node, that node has type `Bool`, and its value on a string is the graph's verdict. Section 15 gives the full definition.

3.5 Totality and the bottom value

Some operations are undefined on some inputs: the first character of the empty string, division by zero, the character at index 9 of a 3-character string. In ordinary programming these raise exceptions. In a synthesiser this is a bad mechanism, for two reasons: exception handling in the inner loop of a search that evaluates millions of candidates is slow, and "the computation threw" is not a *value*, so it cannot be composed.

The standard fix is to add a distinguished value **bottom**, written ⊥, meaning "undefined". Every operator becomes **total**: it returns a value for every possible input, using ⊥ for the previously-undefined cases. Bottom propagates strictly — if any argument is ⊥, the result is ⊥.

```
HEAD("")        = ⊥
DIV(5, 0)       = ⊥
ADD(⊥, 3)       = ⊥
NOT(⊥)          = ⊥
```

This replaces the `Flag` field that every node carried in the original document. `Flag` was defined as "whether the computation succeeded"; that is now simply the test `v ≠ ⊥`, and it does not need to be stored.

At the top level, a graph whose output is ⊥ is treated as classifying the string **negative**. This is a deliberate convention, and it must be stated because it affects what the learner can express.

3.6 Purity

An operator is **pure** if its result depends only on its argument values — no hidden state, no randomness, no side effects. Every operator in our DSL must be pure.

This sounds like a stylistic preference. It is not: it is a load-bearing requirement, because the central optimisation in the search (observational equivalence, Section 4.3) is *only correct* if operators are pure. If someone later adds a stateful or randomised operator, that optimisation silently becomes unsound. This should be documented at the operator interface.

4. Searching for a hypothesis

We now turn to question 3: how to find a good hypothesis. The problem is a combinatorial search over a space that grows roughly exponentially with expression size.

4.1 The size of the space

If there are `|Ops|` operators of arity at most `a`, the number of well-typed expressions of size `k` grows roughly like `|Ops|^k` (types and arity reduce the constant, but not the exponential shape). With 30 operators, expressions of size 8 already number in the hundreds of billions. Blind enumeration is hopeless without pruning, and blind *random* sampling — which is what the original document proposed — is worse, because it repeatedly resamples the same behaviours and has no way to recognise progress.

Three families of technique make this tractable. We recommend the first, support the second, and mention the third for completeness.

4.2 Bottom-up enumerative synthesis

The idea is to build expressions from the leaves upward, in order of increasing size, keeping a **bank** of everything built so far organised by type.

- Size 1: all constants and `INPUT`.
- Size 2: every operator applied to size-1 arguments of the right types.
- Size `k`: every operator applied to argument tuples whose sizes sum to `k − 1`.

Because expressions are produced in strictly increasing size order, the *first* consistent hypothesis found is guaranteed to be a smallest one. That is a genuine optimality guarantee, and it is exactly what Occam's razor asks for. Contrast this with a randomised search, which offers no such guarantee.

This technique is the standard solver architecture in the **syntax-guided synthesis** (SyGuS) literature. On its own it still blows up. What makes it practical is the next idea.

4.3 Observational equivalence

Two expressions are **observationally equivalent** with respect to a set of inputs if they produce identical values on every one of those inputs. For example, on the five training strings of our running example, `LE(MAXRUN(INPUT,'1'), 2)` and `EQ(MAXRUN(INPUT,'1'), 1)` produce exactly the same five booleans. They are different programs, but on this data they are indistinguishable.

The optimisation: **keep only one representative of each observed behaviour.** When a newly built expression's behaviour vector has already been seen, throw it away.

This is sound — meaning it cannot cause us to miss a smallest consistent hypothesis — precisely because operators are pure (Section 3.6). If `e` and `e'` agree on all inputs of interest, then `op(e, x)` and `op(e', x)` also agree, so the discarded expression could never have led anywhere the kept one cannot reach.

The effect is dramatic, and it is worth seeing the arithmetic. With `m` training examples, a `Bool`-valued expression has at most `3^m` distinguishable behaviours (each of `m` positions is `true`, `false`, or ⊥). For `m = 5`, that is at most **243 behaviourally distinct boolean expressions in existence**, regardless of how many syntactic forms there are. The bank cannot grow beyond that, so a search space that was astronomically large collapses to something a laptop finishes in milliseconds.

Two caveats to record honestly:

- Dedup is relative to the *training inputs only*. The representative that is kept may generalise better or worse than the ones discarded. Tie-break by size, then by a fixed deterministic rule, and log which representative was chosen.
- Small `m` is what makes the collapse so effective, and small `m` is also what makes the result untrustworthy (Section 2.7). The two facts are the same fact seen from opposite sides.

4.4 Evolutionary search

The alternative family keeps a **population** of candidate hypotheses and improves it over generations:

1. Score every individual with the objective function (Section 2.6).
2. Select parents, biased toward high scores — **tournament selection** picks `t` individuals at random and keeps the best, which is simple and works well.

3. Produce children by **mutation** (change an operator, rewire an edge, insert or delete a node) and **crossover** (splice a subgraph from one parent into another).
4. Replace the population and repeat.

Applied to expression trees this is **genetic programming** (GP). When types constrain which mutations and crossovers are legal, it is **strongly typed GP**. When the representation is a fixed grid of nodes with wiring indices — which is very close to the representation the original document described — it is **Cartesian genetic programming** (CGP).

The essential ingredient the original document lacked is **selection pressure**. Appending any type-correct node is a random walk. Appending nodes and then *keeping the ones that score better* is search. That single change converts the proposed method into a viable algorithm.

Evolutionary search gives no optimality guarantee and no bound on runtime, but it degrades gracefully as the operator set grows, where enumeration falls off a cliff. Behaviour-vector caching (Section 4.3) can be reused inside it to reject duplicates cheaply.

4.5 Constraint-based synthesis

For completeness: one can encode "is there a program of size $\leq k$ consistent with D?" as a SAT or SMT formula and hand it to a solver. This is powerful and gives completeness up to the size bound, but the encoding is intricate and string operations encode poorly. It is not recommended here, and is mentioned so the reader knows the option exists.

4.6 Comparison

Approach	Optimality	Scales with DSL size	Effort to implement	Recommended role
Bottom-up enumeration + dedup	Smallest consistent hypothesis, guaranteed	Poorly	Low	Primary method
Strongly typed GP / CGP	None	Well	Medium	Alternative for large operator sets
SAT/SMT encoding	Complete up to size bound	Moderately	High	Not recommended
Random growth (original)	None	—	Low	Baseline only, to demonstrate the gap

5. Assembling a concept from parts

Even with the techniques above, searching directly for a large hypothesis is hard, because difficulty grows with expression size. A useful decomposition is to search for *small pieces* and then combine them by a different method.

5.1 Two layers

- **Layer 1 — feature synthesis.** Enumerate many small `Bool`-valued expressions. Each one is a **predicate**: a simple, interpretable test such as "starts with 1", "length is even", "count of 1s is at most 2".
- **Layer 2 — concept assembly.** Treat each predicate as a binary feature, and learn a boolean combination of them that separates positives from negatives.

This works because a concept that needs a size-12 expression may well be a conjunction of three size-4 predicates — and finding three size-4 expressions is vastly cheaper than finding one size-12 expression. It also improves interpretability, since the output reads as a rule over named tests.

Layer 2 turns the tail of the problem into ordinary attribute-value classification over binary features, which is exactly standard data mining machinery.

5.2 Disjunctive normal form

A formula in **disjunctive normal form** (DNF) is an OR of ANDs:

$$(p_1 \wedge \neg p_2) \vee (p_3 \wedge p_4 \wedge \neg p_5)$$

Each bracketed group is a **term** or **rule**; each p_i or $\neg p_i$ is a **literal**. Every boolean function can be written in DNF, so nothing is lost by restricting Layer 2 to this form. DNF is also the natural reading of a rule set: "an instance is positive if it matches rule 1, or rule 2, or …".

5.3 Sequential covering

The classical way to learn a DNF from examples is **sequential covering**, the algorithm behind CN2, RIPPER, and the rule-learning literature generally:

```
Rules ← empty
while some positive example is still uncovered:
    learn one term that covers some uncovered positives
        and no negatives at all
    add it to Rules
    mark the positives it covers as covered
return the OR of Rules
```

Terms are grown greedily. Start from the empty term (which covers everything, including all negatives), then repeatedly add the literal that best improves the term, until no negatives remain covered.

"Best" is measured by a **gain heuristic**. Two standard choices:

- **Information gain**, familiar from decision-tree induction: the reduction in entropy of the class distribution achieved by adding the literal.
- **FOIL gain**, designed for rule learning, which weights the improvement in precision by the number of positives retained:

$$\text{gain}(p) = t \cdot \left(\log_2(P_1/(P_1+N_1)) - \log_2(P_0/(P_0+N_0)) \right)$$

where P_0, N_0 are the positives and negatives covered before adding p, P_1, N_1 after, and t is the number of positives still covered after. FOIL gain is the better default here because it explicitly resists rules that achieve purity by covering almost nothing.

5.4 Decision trees as an alternative

Running a decision-tree learner (ID3, C4.5) over the same binary predicate features is an equally valid Layer 2, and the resulting tree converts directly to DNF by reading off the paths to positive leaves. It is worth implementing both and comparing, since the comparison is cheap and informative.

5.5 Pruning

Both rule sets and trees will happily grow until they memorise the training data — the rote-learner failure of Section 2.4 reappearing at Layer 2. The guard is the same: apply the MDL cost of Section 2.6 to the assembled hypothesis, and accept an extra term only if it pays for itself.

6. Formal-language background

The target concepts in this project are sets of strings, so the theory of formal languages tells us a great deal about what we are attempting. This section is likely to be the most familiar part of Part I.

6.1 Alphabets, strings, languages

Σ is a finite **alphabet**, Σ^* the set of all finite strings over it, and a **language** is any subset of Σ^*. A concept over strings and a language are the same object under two names. The original document fixed $\Sigma = \{0, 1\}$; the revised specification allows Σ to be declared in the input file, at essentially no implementation cost.

6.2 Regular languages and finite automata

A language is **regular** if it is recognised by a deterministic finite automaton (DFA) — a machine with finitely many states that reads the string left to right and accepts if it ends in an accepting state. Regular languages are closed under union, intersection, and complement, and are exactly the languages describable by regular expressions.

The **pumping lemma** gives the standard tool for proving a language is *not* regular: any sufficiently long string in a regular language has a middle section that can be repeated arbitrarily while staying in the language. A language requiring unbounded counting of two quantities against each other fails this test.

6.3 Where the target concepts sit

This is worth tabulating, because it determines what the DSL must be able to express and which baselines apply.

Concept	Class	Minimal DFA	Notes
Odd number of 1s	Regular	2 states	Parity; requires `MOD` in an expression language
At most 2 consecutive 1s	Regular	4 states	Requires run-length or a fold
Count of 1s is more than 5	Regular	7 states	Requires a `>` comparison
More 1s than 0s	**Not regular**	—	Unbounded counter; context-free

Three of the four are regular. The fourth is not: "more 1s than 0s" needs to track a difference that can grow without bound, which no finite-state machine can do. It is context-free, recognisable by a pushdown automaton, and it is a useful deliberate demonstration that an expression language with integer arithmetic can reach *outside* the regular class — a DFA-based learner cannot express it at all. This is the strongest argument for the expression-language approach and should be stated as such in the project's motivation.

6.4 Grammatical inference

Learning a language from labelled examples is a mature field called **grammatical inference**, and its algorithms are the natural baselines.

- **RPNI** (Regular Positive and Negative Inference) builds a tree-shaped automaton that accepts exactly the positive examples, then repeatedly tries to **merge** pairs of states, rejecting any merge that would cause a negative example to be accepted. It runs in polynomial time and identifies any regular language in the limit given a characteristic sample.
- **EDSM** (Evidence-Driven State Merging) is the same idea with a heuristic that orders candidate merges by how much evidence supports them. It won the Abbadingo One competition and remains a strong practical baseline.
- **L*** (Angluin) learns a DFA using **membership queries** ("is this string in the language?") and **equivalence queries** ("is this automaton correct?"). It is the canonical active-learning algorithm and is exactly what Section 2.7 gestured at.

RPNI must be implemented as a baseline. For three of the four stated target concepts it solves the problem in polynomial time with theoretical guarantees. Any proposed method should be compared against it, and the project's justification should rest on the things RPNI cannot do — namely non-regular concepts and human-readable output.

6.5 Why still use an expression language?

Given the above, the case for this project rests on three claims, all of which should be stated in the introduction and tested in the evaluation:

1. **Interpretability.** `MAXRUN(s, '1') ≤ 2` is immediately readable. A 4-state transition table is not.

2. **Reach beyond regular.** Integer arithmetic over string statistics escapes the regular class, as "more 1s than 0s" demonstrates.
3. **Extensibility.** Adding a new concept family means adding one operator, not redesigning the learner.

6.6 A note on expressive power within the DSL

A finite expression DAG with no recursion or iteration cannot examine an unbounded portion of a string one symbol at a time. It can only inspect a *bounded* prefix or suffix (via `PREFIX`, `HEAD`, `CHARAT`) plus whatever the aggregating operators summarise (`LEN`, `COUNT`, `MAXRUN`, `DEC`). So the expressible class is, roughly, boolean combinations of arithmetic relations over string statistics.

That is a real limitation, and it should be documented rather than discovered. If concepts requiring genuine sequential processing are needed, the clean extension is to add a single `DFA` operator of type `Str → Bool` whose transition table is itself learned by RPNI. This makes automaton learning a *primitive* inside the expression language, giving the union of both approaches' expressive power. It is recommended as a stretch goal, not as part of the core.

7. How results should be measured

A learner that reports its accuracy on the data it was trained on is reporting nothing. This section fixes the protocol.

7.1 Train, validation, and test

- The **training set** is what the learner sees.
- The **validation set** is used to choose among candidate hypotheses (for example, to select the MDL trade-off).
- The **test set** is used exactly once, at the end, to report performance. If it influences any decision, it has become a validation set and no longer measures generalisation.

Because our examples come from a known rule, test data can be **generated**: sample strings from `Σ^≤n`, label them with the ground-truth oracle, and hold them out. This removes the usual excuse that there are too few examples to split.

7.2 Cross-validation

Where data are genuinely scarce, **k-fold cross-validation** splits the examples into `k` parts, trains on `k−1` and tests on the remaining one, and averages over all `k` choices. It uses every example for both roles without ever testing on training data. Standard, and worth having in the harness.

7.3 Metrics

Report accuracy, precision, recall, and F1, plus the full confusion matrix. For a rule-learning system also report **hypothesis size**, since a small accurate hypothesis and a large accurate one are very different outcomes and only one of them has met the project's stated goal.

7.4 Boundary sampling

Uniform random sampling is a weak test for threshold concepts. If the rule is "at most 2 consecutive 1s", most random strings are far from the boundary and easy. The informative cases have a maximal run of exactly 2 or exactly 3.

Generate a **boundary set** deliberately: strings whose relevant statistic sits at, just below, and just above the threshold. This is where competing hypotheses disagree, so it is where the numbers become meaningful. Report boundary-set accuracy separately from uniform accuracy.

7.5 Reproducibility

Any randomised component must take an explicit seed, and the seed must be logged with every run alongside the operator set, size limits, and objective parameters. A randomised result that cannot be reproduced is not a result.

7.6 Baselines and ablations

Report at minimum:

- **Majority class** — the accuracy of always predicting the more common label. If a method cannot beat this, it has learned nothing.
- **RPNI** — Section 6.4.
- **Random growth** — the original document's method, retained as a baseline to quantify what the redesign bought.

And measure cost, not only accuracy: candidates evaluated, wall-clock time, peak bank size. A method that is 2% more accurate and 1000× slower is a different proposition from one that is 2% more accurate for free.

8. Glossary

Active learning — learning where the algorithm chooses which instances to be labelled, rather than receiving a fixed sample.

Arity — the number of parameters an operator takes.

Bottom (⊥) — a distinguished value meaning "undefined", used to make every operator total.

Behaviour vector — the tuple of values an expression produces across the training inputs; the key used for observational-equivalence deduplication.

Concept — a subset of the instance space; the thing being learned.

Consistent — classifying every training example correctly.

DAG — directed acyclic graph; the shape of a hypothesis with shared sub-expressions.

Denotational semantics — a definition of what each syntactic construct means as a mathematical function.

DNF — disjunctive normal form; an OR of ANDs of literals.

DSL — domain-specific language; the restricted language hypotheses are written in.

Grammatical inference — the field concerned with learning formal languages or automata from examples.

Hypothesis space — the set of all hypotheses a learner can produce.

Inductive bias — the assumptions a learner uses to choose among hypotheses that fit the data equally well.

Instance space — the set of all objects that could be classified; here Σ*.

MDL — Minimum Description Length; an objective balancing hypothesis size against data fit.

Membership query — asking an oracle whether a chosen instance is positive.

Observational equivalence — producing identical values on all inputs of interest; the basis of the main search optimisation.

Overfitting — fitting the training data at the expense of generalisation.

Pure — depending only on argument values; no state, randomness, or side effects.

RPNI — a polynomial-time DFA-learning algorithm based on state merging.

Sequential covering — the greedy algorithm for learning a rule set one rule at a time.

Signature — an operator's parameter types and result type.

SyGuS — syntax-guided synthesis; the framework of program synthesis constrained by a grammar.

Total — defined on every input.

VC dimension — a measure of the expressive richness of a hypothesis space, used in sample-complexity bounds.

PART II — REVISED SPECIFICATION

9. Scope and goals

9.1 Purpose

The software learns a human-readable rule that separates positively- from negatively-labelled strings, and reports that rule together with an honest assessment of how much confidence the data support.

9.2 Functional requirements

- **F1.** Read one or more test cases from a text file in the format of Section 10.
- **F2.** For each test case, produce a hypothesis in the language of Sections 11 to 15, or report that no consistent hypothesis exists within the configured search budget.
- **F3.** Render each hypothesis in three forms: a readable formula, a graph (Graphviz DOT), and a machine-readable JSON serialisation.
- **F4.** Classify arbitrary unseen strings using a learned hypothesis.
- **F5.** Report the evaluation metrics of Section 19.
- **F6.** Report whether the training set identifies the hypothesis, by listing all equally-small consistent alternatives found (Section 19.4).

9.3 Non-functional requirements

- **N1.** Every randomised component takes an explicit seed; runs are exactly reproducible from the seed and the configuration.
- **N2.** The search strategy is replaceable without modifying the hypothesis language, the evaluator, or the objective function.
- **N3.** Adding a new operator requires registering it in one place and no changes to the search or evaluation code.
- **N4.** Every search is bounded by an explicit budget (maximum expression size, maximum bank size, wall-clock timeout) and terminates.

9.4 Explicit non-goals

- Noisy or mislabelled data. Labels are assumed exact.
- Concepts requiring unbounded sequential processing, unless the optional `DFA` operator of Section 13.9 is enabled.
- Alphabets larger than a few dozen symbols.

10. Input format

10.1 Grammar

The original document described the format only by example. Here it is as EBNF. Encoding is UTF-8; line endings are LF or CRLF.

```
file          = { blank | comment | testcase } ;

comment       = "#" , { any-char - newline } , newline ;
blank         = { space | tab } , newline ;

testcase      = header , { metadata } , { example } ;

header        = "=====" , spaces , "Test Case" , spaces , identifier ,
                spaces , "=====" , newline ;

metadata      = key , ":" , spaces , value , newline ;
```

```
key           = "alphabet" | "description" | "oracle" | "author" ;

example       = label , spaces , string , newline ;
label         = "+" | "-" ;
string        = { symbol } ;                  (* possibly empty *)
symbol        = ? any character in the declared alphabet ? ;

identifier    = letter-or-digit , { letter-or-digit | "_" | "-" } ;
spaces        = space , { space | tab } ;
```

10.2 Example

```
# Concepts over binary strings, revision 2 format.

===== Test Case 1024 =====
alphabet: 01
description: The longest run of consecutive 1s is at most 2.
oracle: maxrun(s,'1') <= 2

+ 1001
+ 00001
- 11100
+ 10
- 111
+ 11
- 01110
+ 110011
```

10.3 Rules and edge cases

These were undefined in the original and each is a real source of bugs.

Situation	Required behaviour
Line beginning with #	Comment; ignored
Blank line	Ignored; not a test-case separator
`alphabet` key absent	Defaults to `01`
Symbol not in the declared alphabet	Fatal parse error naming the line number
Empty string after a label	Legal; represents ε
Duplicate string with the same label	Deduplicated, with a warning
Duplicate string with conflicting labels	Fatal error; the data are inconsistent
Example line before any header	Fatal parse error
Test case with no positives or no negatives	Warning; the trivial hypothesis will win
`oracle` key present	Used to generate held-out test data (Section 19)

10.4 Label separation — a mandatory correction

The original document specified that the Root node *"always takes the full input line as input"*. Taken literally, the graph would receive `"+ 1001"` including the label. The one-node hypothesis `STARTSWITH(INPUT, "+")` would then be a perfect classifier of every test case, and the system would learn nothing while appearing to succeed.

This is a textbook instance of **label leakage** — the target variable reaching the model through a feature. The parser must therefore split each line into `(label, string)` and construct the training set such that the label is reachable only by the objective function, never by the evaluator. The `Example` record enforces this by exposing the string and the label through separate accessors, and the evaluator's signature takes a `String`, not an `Example`.

11. Type system

Four types:

Type	Domain	Notes
`Str`	Σ^*	The input and derived strings
`Char`	Σ	Single symbols
`Int`	$\mathbb{Z}$ (arbitrary precision)	See 11.1
`Bool`	`{true, false}`	

Plus the value $\perp$ (Section 3.5), which inhabits every type.

An expression is **well-typed** if every operator's arguments match its declared parameter types positionally. A hypothesis is well-typed if all its expressions are and its output node has type `Bool`. Type-checking is a single traversal and must be run as an invariant check on every constructed hypothesis, not only on user input.

11.1 Numeric representation

The original specified "Double". This is wrong for `DEC`: the decimal value of a 60-symbol binary string exceeds `2^53` and a double silently loses precision, so $\texttt{DEC}(s_1) = \texttt{DEC}(s_2)$ can be true for distinct strings. Since the whole system reasons about exact equality, silent precision loss would produce wrong answers with no error.

`Int` is therefore arbitrary precision. `DEC` additionally returns $\perp$ for inputs longer than a configured `MAX_DEC_LEN` (default 64), to bound memory. There is no floating-point type; nothing in the domain requires one, and removing it removes a class of comparison bugs.

12. Values and bottom

```
Value = Str(s) | Char(c) | Int(n) | Bool(b) | ⊥
```

Rules:

1. Every operator is total: it returns a `Value` for every combination of argument values.
2. ⊥ propagates strictly. If any argument is ⊥, the result is ⊥. There is no operator that inspects ⊥, and in particular there is no `IS_DEFINED` — allowing one would let the learner exploit undefinedness as a feature, which is uninterpretable.
3. A hypothesis whose output is ⊥ on a string classifies that string **negative**.

Rule 3 is a design decision with consequences. It means undefinedness is not distinguishable from a genuine negative at the top level, which keeps the output type binary and the semantics simple, at the cost of letting a hypothesis "succeed" partly by failing. The objective function's size penalty is the counterweight. This trade-off should be revisited if it causes trouble in practice.

The `Flag` field of the original document is removed; it is now $v \neq \bot$ and needs no storage.

13. Operator set

Each operator has a symbol, a parameter list, and a result type. Constants are operators of arity zero.

13.1 Structural

Symbol	Signature	Meaning
`INPUT`	`() → Str`	The input string, label already stripped

The `Root` node of the original is replaced by `INPUT`. The `End` node is removed entirely: a hypothesis terminates because its output node has type `Bool`, not because a marker node says so. This removes the ambiguity about how multiple `End` nodes would have combined.

13.2 Constants

Symbol	Signature	Domain
`INT[n]`	`() → Int`	$n \in \{0, \ldots, K\}$, K configurable (default 8)
`CHR[c]`	`() → Char`	$c \in \Sigma$
`STR[w]`	`() → Str`	w from a configured pool; see 13.10
`TRUE`, `FALSE`	`() → Bool`	

13.3 String to string

Symbol	Signature	Meaning	⊥ when
`TRIMHEAD`	`Str × Int → Str`	drop n symbols from the left	$n < 0$
`TRIMTAIL`	`Str × Int → Str`	drop n symbols from the right	$n < 0$
`PREFIX`	`Str × Int → Str`	first n symbols	$n < 0$

Symbol	Signature	Meaning	⊥ when
SUFFIX	Str × Int → Str	last n symbols	n < 0
REVERSE	Str → Str	reverse	never

TRIMHEAD/TRIMTAIL return the empty string when n exceeds the length, rather than ⊥; this keeps them total in the common case and avoids ⊥-flooding the bank.

13.4 String to integer

Symbol	Signature	Meaning	⊥ when
LEN	Str → Int	length	never
COUNT	Str × Char → Int	occurrences of a symbol	never
COUNTSUB	Str × Str → Int	non-overlapping occurrences of a substring	argument is empty
MAXRUN	Str × Char → Int	length of the longest run of a symbol	never
INDEXOF	Str × Str → Int	index of first occurrence	not present, or empty needle
DEC	Str → Int	value read as base-|Σ|	length exceeds MAX_DEC_LEN

MAXRUN and COUNTSUB are the two additions that make run-length and substring concepts expressible at all. They did not exist in the original, which is why Test Case 1024's own stated concept was outside its hypothesis space.

13.5 String to character

Symbol	Signature	Meaning	⊥ when
HEAD	Str → Char	first symbol	empty string
LAST	Str → Char	last symbol	empty string
CHARAT	Str × Int → Char	symbol at index n, 0-based	out of range

13.6 String predicates

Symbol	Signature	Meaning
STARTSWITH	Str × Str → Bool	prefix test
ENDSWITH	Str × Str → Bool	suffix test
CONTAINS	Str × Str → Bool	substring test
STREQ	Str × Str → Bool	equality

These take a `Str` second argument, not a `Char`. The original's `Has` node took only a single character, which is why "contains `11`" was inexpressible. A single character is still reachable via a length-1 `STR` constant.

13.7 Arithmetic

Symbol	Signature	⊥ when
`ADD`, `SUB`, `MUL`	`Int × Int → Int`	never
`DIV`, `MOD`	`Int × Int → Int`	divisor is 0
`MIN`, `MAX`	`Int × Int → Int`	never
`ABS`	`Int → Int`	never

`SUB` and `MOD` are new. `MOD` is what makes parity concepts expressible; `SUB` is what makes difference-based concepts expressible without contortion. `DIV` and `MOD` use floored (Euclidean) semantics, fixed explicitly so that negative operands behave predictably across implementation languages.

13.8 Comparison and boolean

Symbol	Signature
`EQ`, `NEQ`, `LT`, `LE`, `GT`, `GE`	`Int × Int → Bool`
`CHREQ`	`Char × Char → Bool`
`AND`, `OR`, `XOR`	`Bool × Bool → Bool`
`NOT`	`Bool → Bool`

The absence of these from the original is the largest single expressiveness gap it had: with equality as the only comparison and no boolean connectives, none of the four concepts named in the original's own introduction could be written down.

13.9 Optional: learned automaton

Symbol	Signature	Meaning
`DFA[M]`	`Str → Bool`	acceptance by automaton `M`

Disabled by default. When enabled, `M` is produced by RPNI on the training set and the whole automaton is treated as a single primitive of size `|M|` (its state count), so the size penalty of Section 14.3 charges fairly for its complexity. This is the extension point discussed in Section 6.6 and should be attempted only after the core system works.

13.10 Guarding `STR` constants against memorisation

`STR` constants are the mechanism by which a rote learner could arise: with `STREQ(INPUT, "1001")` available, `OR`-ing together one equality per positive example is always consistent and always useless.

Two guards, both required:

1. **The STR constant pool excludes any string that appears in the training set**, and is limited to strings of length at most `L_str` (default 3).
2. The size penalty of the objective function makes an `n`-example memorisation cost at least `4n` nodes, which loses to any genuine rule of modest size.

The evaluation harness must include a regression test confirming that the learner does *not* produce a memorising hypothesis on a small training set.

14. Hypothesis representation

14.1 Structure

A **concept graph** `G` is a tuple `(N, op, args, out)`:

- `N` a finite set of nodes.
- `op : N → Operator`.
- `args : N → N*` the ordered list of argument nodes, with `|args(n)| = arity(op(n))`.
- `out ∈ N` the single output node.

14.2 Invariants

Checked on construction and after every mutation:

- **I1 — Acyclicity.** The relation `n → args(n)` has no cycles.
- **I2 — Well-typedness.** For every `n` and every position `i`, the result type of `op(args(n)[i])` equals the `i`-th parameter type of `op(n)`.
- **I3 — Output type.** `op(out)` has result type `Bool`.
- **I4 — Reachability.** Every node is reachable from `out`. Unreachable nodes are dead code and are stripped, so that `size` measures the actual hypothesis.

14.3 Size

```
size(G) = |{ n ∈ N : n reachable from out }|
```

Shared sub-expressions are counted once. This makes the DAG representation genuinely cheaper than the equivalent tree and gives the search a reason to prefer sharing — which is also what a human reader prefers.

15. Semantics

This section defines what a hypothesis *means*. It is the piece the original document was missing, and everything else depends on it.

15.1 Definition

For a concept graph G and a string $s \in \Sigma^*$, define $\mathrm{eval} : N \to \mathrm{Value}$ by structural recursion:

$$
\begin{aligned}
\mathrm{eval}(n) &= s && \text{if } \mathrm{op}(n) = \mathrm{INPUT} \\
&= k && \text{if } \mathrm{op}(n) \text{ is the constant } k \\
&= \bot && \text{if any } \mathrm{eval}(\mathrm{args}(n)[i]) = \bot \\
&= \mathrm{op}(n).\mathrm{apply}(\, \mathrm{eval}(\mathrm{args}(n)[1]), \ldots, \mathrm{eval}(\mathrm{args}(n)[k]) \,) && \text{otherwise}
\end{aligned}
$$

eval is well-defined because of invariant I1: the recursion terminates on a finite acyclic graph. The **denotation** of G is then

$$
\begin{aligned}
[\![G]\!](s) &= \mathrm{true} && \text{if } \mathrm{eval}(\mathrm{out}) = \mathrm{Bool}(\mathrm{true}) \\
&= \mathrm{false} && \text{otherwise (covering } \mathrm{Bool}(\mathrm{false}) \text{ and } \bot)
\end{aligned}
$$

so $[\![G]\!] : \Sigma^* \to \{\mathrm{true}, \mathrm{false}\}$ is total. This is the function that classifies strings, and it is defined for every string, not only those seen in training.

15.2 Evaluation with sharing

A direct recursive implementation would re-evaluate a shared node once per parent, costing time exponential in depth. Memoise within a single evaluation:

```
EVAL(G, s):
    memo ← empty map
    return VALUE(G.out, s, memo)

VALUE(n, s, memo):
    if n ∈ memo:
        return memo[n]
    if op(n) = INPUT:
        v ← Str(s)
    else if arity(op(n)) = 0:
        v ← constant value of op(n)
    else:
        vs ← [ VALUE(a, s, memo) for a in args(n) ]
        v  ← ⊥  if any v_i = ⊥  else  op(n).apply(vs)
    memo[n] ← v
    return v

CLASSIFY(G, s):
    return (EVAL(G, s) = Bool(true))
```

With memoisation, evaluation is $O(\mathrm{size}(G))$ operator applications.

15.3 Behaviour vectors

For a list of examples `D = [(s₁,ℓ₁), …, (s_m,ℓ_m)]`, the **behaviour vector** of an expression `e` is

```
beh(e, D) = ( eval_e(s₁), …, eval_e(s_m) )
```

This is the key used for observational-equivalence deduplication (Section 4.3) and the unit of currency in the search. Note it is computed from the *strings only* — the labels never enter it. That separation is what makes Section 10.4's leakage guard structural rather than a matter of discipline.

15.4 Rendering

Every hypothesis must be renderable as a readable formula, which is a straightforward traversal with infix printing for binary operators and `let`-bindings for nodes with more than one parent:

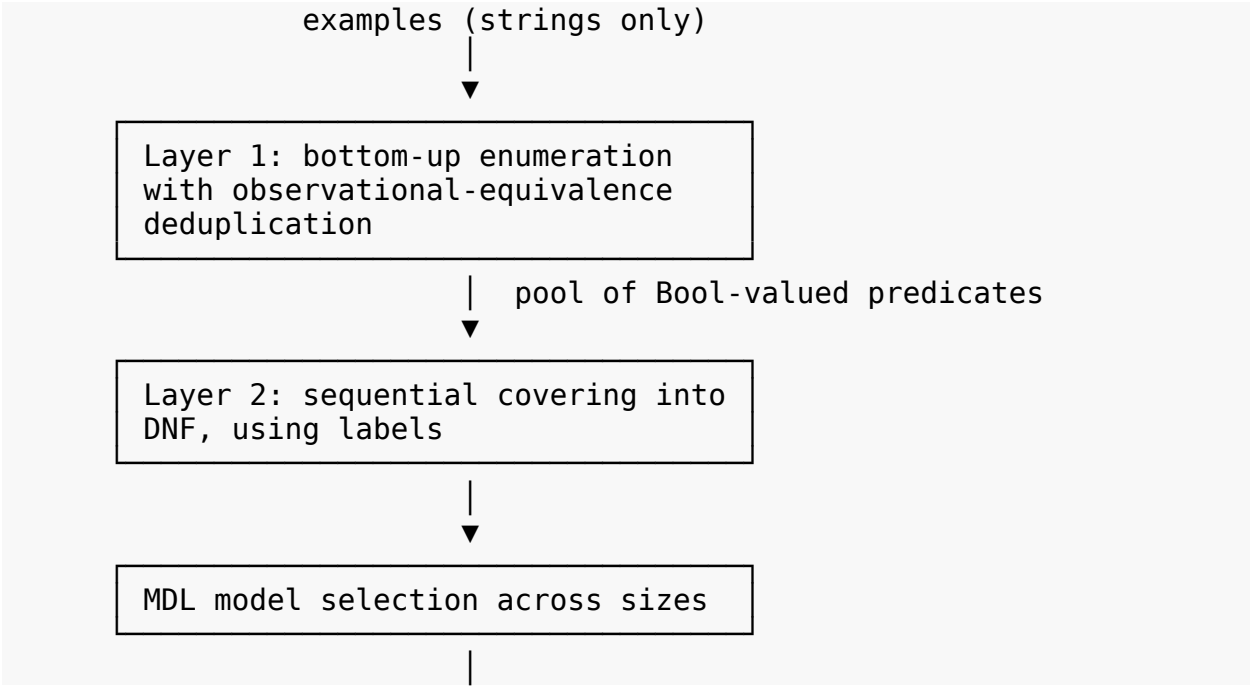
```
let r = MAXRUN(INPUT, '1') in  r <= 2
```

This rendering is the primary deliverable of the whole system and should be tested as carefully as the evaluator.

16. The learning algorithm

16.1 Overview

Two layers, as motivated in Section 5.1.



```
                          ▼
                  hypothesis + report
```

Layer 1 does not see labels at all. Layer 2 is the only place labels are consulted.

16.2 Layer 1 — enumeration

```
ENUMERATE(S, Ops, maxSize, poolCap):
    # S: the training strings, labels excluded
    # Bank: Type → list of (expr, size, behaviour)
    # Seen: set of (Type, behaviour)

    Bank ← empty ; Seen ← empty

    for each operator c in Ops with arity 0:
        INSERT(Bank, Seen, expr = c, size = 1, S)

    for size = 2 to maxSize:
        for each operator op in Ops with arity k ≥ 1:
            for each tuple (e₁, …, e_k) such that
                    eᵢ ∈ Bank[ op.paramType(i) ]  and
                    1 + Σ size(eᵢ) = size:
                e ← op(e₁, …, e_k)
                b ← [ EVAL(e, s) for s in S ]
                if every component of b is ⊥:
                    continue                        # degenerate
                if (op.resultType, b) ∈ Seen:
                    continue                        # observationally
equivalent
                INSERT(Bank, Seen, e, size, S)
        if |Bank| > poolCap:
            PRUNE(Bank)                             # keep smallest per
behaviour
    return Bank[Bool]

INSERT(Bank, Seen, e, size, S):
    b ← [ EVAL(e, s) for s in S ]
    Bank[resultType(e)].append( (e, size, b) )
    Seen.add( (resultType(e), b) )
```

Three points on why this is correct and why it is fast.

- The size-partitioned inner loop (`1 + Σ size(eᵢ) = size`) is what enforces strict increasing size order. That order is what makes the first consistent hypothesis provably smallest.
- The `Seen` check is the observational-equivalence optimisation. Its soundness depends on operator purity (Section 3.6).
- The all-⊥ check discards expressions that are undefined everywhere; they can never contribute, since ⊥ propagates.

Complexity. Let `m = |S|`. The number of distinguishable `Bool` behaviours is at most `3^m`; for `Int` it is bounded by the number of distinct value tuples actually produced, which in practice is small. The bank therefore saturates rather than growing exponentially, and each level costs `O(|Ops| · |Bank|^a · m)` evaluations where `a` is the maximum arity. For `m = 8` and `maxSize = 8` this is comfortably interactive.

Section 21 refines this bank into a structure whose entries are equivalence classes carrying the productions that build them, which is what allows a value derived for one purpose to serve as an argument elsewhere. The enumeration order above, and therefore the smallest-first guarantee, is unchanged by that refinement.

16.3 Layer 2 — sequential covering

```
LEARN_DNF(Preds, Pos, Neg):
    Rules ← empty list
    Uncovered ← Pos
    while Uncovered is not empty:
        term ← LEARN_TERM(Preds, Uncovered, Neg)
        if term = NONE:
            return NONE                  # not separable with this
pool
        Rules.append(term)
        Uncovered ← Uncovered \ { s ∈ Uncovered : term(s) = true }
    return OR(Rules)

LEARN_TERM(Preds, Pos, Neg):
    term ← TRUE
    covP ← Pos ; covN ← Neg
    while covN is not empty:
        best ← argmax over p ∈ (Preds ∪ { ¬p : p ∈ Preds })
                    of FOIL_GAIN(p, covP, covN)
        if FOIL_GAIN(best, covP, covN) ≤ 0:
            return NONE
        term ← AND(term, best)
        covP ← { s ∈ covP : best(s) = true }
        covN ← { s ∈ covN : best(s) = true }
    return term

FOIL_GAIN(p, covP, covN):
    P₀ ← |covP| ; N₀ ← |covN|
    P₁ ← |{ s ∈ covP : p(s) }| ; N₁ ← |{ s ∈ covN : p(s) }|
    if P₁ = 0: return −∞
    return P₁ · ( log₂(P₁/(P₁+N₁)) − log₂(P₀/(P₀+N₀)) )
```

Ties in the `argmax` are broken by smaller predicate size, then by a fixed deterministic order, so that the result does not depend on hash iteration order. This matters for requirement N1.

16.4 Top level and model selection

```
LEARN(testCase, Ops, maxSize):
    (Pos, Neg) ← PARSE(testCase)           # labels separated; Section
```

```
10.4
    S    ← strings(Pos) ∪ strings(Neg)
    best ← NONE

    for k = 1 to maxSize:
        Preds ← ENUMERATE(S, Ops, k, poolCap)

        # a single predicate suffices?
        C ← { p ∈ Preds : consistent(p, Pos, Neg) }
        if C is not empty:
            return SELECT(C)              # smallest; report all of C

        H ← LEARN_DNF(Preds, Pos, Neg)
        if H ≠ NONE and COST(H) < COST(best):
            best ← H
        if best ≠ NONE and k ≥ size(best):
            break                         # nothing smaller can exist
    return best

COST(H) = errors(H, D) · log₂|D| + size(H) · log₂|Ops|
```

The early return when a single predicate is consistent is not just an optimisation: because enumeration proceeds in size order, that predicate is guaranteed to be a smallest consistent hypothesis in the whole space. The loop returns the *set* `C` rather than one element, because reporting all equally-small consistent hypotheses is requirement F6 — see Section 19.4 and Appendix A for why this matters more than it might appear.

16.5 Alternative strategy — evolutionary search

Provided as a second implementation of the same interface, for operator sets too large to enumerate.

```
EVOLVE(D, Ops, popSize, generations, seed):
    rng ← RandomSource(seed)
    P   ← [ RANDOM_GRAPH(Ops, rng) for 1..popSize ]
    seenBeh ← empty set
    best ← argmin over P of COST

    for g = 1 to generations:
        P' ← empty
        while |P'| < popSize:
            a ← TOURNAMENT(P, t, rng)
            b ← TOURNAMENT(P, t, rng)
            child ← CROSSOVER(a, b, rng)   with prob p_x, else COPY(a)
            child ← MUTATE(child, Ops, rng) with prob p_m
            if not (I1 ∧ I2 ∧ I3 hold for child): continue
            child ← STRIP_UNREACHABLE(child)
            if beh(child, D) ∈ seenBeh: continue     # cheap duplicate
rejection
            seenBeh.add(beh(child, D))
```

```
            P'.append(child)
        P ← P'
        best ← argmin over (P ∪ {best}) of COST
        if COST(best) = 0: break
    return best

MUTATE options: replace an operator with another of the same
signature;
                rewire one argument edge to another node of the right
type;
                insert a node above a randomly chosen node;
                delete a node, splicing a type-compatible child into
its place.
```

The essential difference from the original document's method is the `TOURNAMENT` step and the `COST` function. Appending any type-correct node is a random walk; appending nodes and keeping those that score better is search.

16.6 Baseline strategy — RPNI

Implemented as a third strategy for comparison (Section 19.5).

```
RPNI(Pos, Neg):
    A ← prefix tree acceptor of Pos          # accepts exactly Pos
    for each pair (q, q') of states in canonical order:
        if q and q' are still distinct:
            A' ← MERGE(A, q, q')             # with determinising
cascade
            if A' accepts no string in Neg:
                A ← A'
    return A
```

When the optional `DFA` operator is enabled, the resulting automaton can be wrapped as a single primitive and used inside the expression language.

17. Software architecture

17.1 Layering

```
┌──────────────────────────────────────────┐
│  cli / harness                           │
├──────────────────────────────────────────┤
│  learn        (ConceptLearner, RuleLearner)
├──────────────────────────────────────────┤
│  search       (strategies)   │  objective │
├──────────────────────────────────────────┤
│  lang         (types, operators, graph, evaluator)
├──────────────────────────────────────────┤
│  io  (parser)          report (DOT, JSON, formula)
└──────────────────────────────────────────┘
```

Dependencies point strictly downward. `lang` knows nothing about learning; `search` knows nothing about the two-layer decomposition; `learn` orchestrates. If any dependency arrow points upward, the design has gone wrong, and in particular `lang` must never import anything that knows what a label is.

17.2 Responsibilities

Component	Responsibility	Must not
`InputParser`	Text to `TestCase`; validation; label separation	Know about hypotheses
`TestCase`, `Example`	Hold data; expose strings and labels separately	Expose the label to the evaluator
`Type`, `Value`	The four types and ⊥	—
`Operator`	Symbol, signature, `apply`; pure and total	Have state or randomness
`OperatorRegistry`	Registration and lookup by result type	Know about search
`Node`, `ConceptGraph`	Structure and invariants I1–I4	Contain evaluation logic
`Evaluator`	`EVAL`, `CLASSIFY`, `beh`; memoisation	See labels
`Objective`	`COST` (Section 16.4)	Modify hypotheses
`SearchStrategy`	`search(examples, budget) → hypotheses`	Assume a particular strategy
`PredicatePool`	Bank of `Bool` expressions keyed by behaviour	—
`RuleLearner`	Sequential covering	Enumerate expressions
`ConceptLearner`	Orchestration, model selection	Contain search logic
`Reporter`	Formula, DOT, JSON, metrics	Compute hypotheses

17.3 Design patterns

- **Strategy** — `SearchStrategy` and `Objective`. This is what makes requirement N2 and the RPNI baseline possible.
- **Registry** — `OperatorRegistry`. This is what makes requirement N3 possible: adding `MAXRUN` is one registration and no other edit.
- **Interpreter / Composite** — `ConceptGraph` and `Evaluator`.
- **Visitor** — rendering, DOT export, size, and type-checking are all traversals and belong outside `Node`, so that adding a new traversal does not modify the graph classes.

17.4 Class diagram

The UML class diagram accompanying this specification shows the classes of Section 17.2 with their principal operations and relationships. One note on it: `RpniBaseline` implements

`SearchStrategy` but produces an automaton rather than an expression graph. It is unified with the others by wrapping the automaton in a single `DFA` operator node (Section 13.9), which is also why that operator is worth having even if the stretch goal is never pursued.

18. Configuration

All of the following are configuration, not constants in the code, and all are logged with every run:

Parameter	Default	Meaning
`seed`	required	RNG seed
`maxSize`	8	Maximum expression size
`poolCap`	200000	Maximum bank entries
`timeout`	60 s	Wall-clock budget per test case
`K`	8	Largest integer constant
`L_str`	3	Longest `STR` constant
`MAX_DEC_LEN`	64	Longest string `DEC` accepts
`operators`	all except `DFA`	Enabled operator set
`strategy`	`enumerate`	`enumerate`, `evolve`, or `rpni`
`layer2`	`covering`	`covering`, `tree`, or `none`

19. Evaluation protocol

19.1 Data

Where a test case declares an `oracle`, generate three additional sets:

- **Uniform test set** — strings sampled uniformly from Σ^≤n, labelled by the oracle.
- **Boundary test set** — strings constructed so the concept's governing statistic sits at, just below, and just above its threshold (Section 7.4).
- **Length-extrapolation set** — strings longer than any in training, to detect hypotheses that only work at training lengths.

Where no oracle is declared, use k-fold cross-validation and say so in the report.

19.2 Metrics

Accuracy, precision, recall, F1, and the confusion matrix, reported separately for each of the three sets above. Plus `size(H)`, since a large accurate hypothesis has not met the project's goal.

19.3 Cost

Candidates evaluated, peak bank size, wall-clock time. Reported for every strategy so that comparisons are meaningful.

19.4 Identifiability report

For each test case, report:

- The number of consistent hypotheses of minimal size found (`|C|` in Section 16.4), and up to ten of them rendered as formulas.
- The sample-complexity estimate from Section 2.7 given the effective hypothesis-space size, with a plain-language verdict such as *"5 examples; at least 40 recommended; this result is not identified by the data."*
- If a ground-truth oracle is available, a **disambiguating example** — a string on which the minimal consistent hypotheses disagree — which is the single most useful example the user could add next.

This last item is the practical payoff of Section 2.7 and turns the tool from a black box into something that tells the user how to improve their data.

19.5 Required comparisons

Every reported experiment includes: majority-class baseline, RPNI, random growth (the original method), and the proposed method. Each with metrics and cost.

20. Output

20.1 Formula

The primary output. One line where possible:

```
Test Case 1024:  MAXRUN(INPUT, '1') <= 2
```

20.2 Graph

Graphviz DOT, with nodes labelled by operator symbol and edges by argument position, so that shared sub-expressions are visible.

20.3 JSON

Machine-readable serialisation of the graph, the configuration, the seed, and the full metrics block, so that results can be regenerated and compared across runs.

20.4 Report

Human-readable summary combining the formula, the metrics of Section 19.2, the cost of Section 19.3, and the identifiability verdict of Section 19.4.

PART IV — CGL CONSTRUCTION

Part II specifies a learner that enumerates expressions and keeps one representative of each observed behaviour. That is sufficient for a tree-shaped search over a fixed training set. It is not sufficient once a value produced on one branch may be an argument to a node on another, which is the defining move of the Concept Graph Learner.

This part specifies the data structure that replaces the representative-per-behaviour bank of Section 16.2: a **value bank** whose entries are equivalence classes rather than expressions, and which records how each class can be produced rather than which expressions belong to it. Section 21 gives the batch construction, which is normative. Section 22 gives incremental ingestion by partition refinement, which is an optional extension — unnecessary when the training set is fixed and known in advance, and required as soon as it is not.

Nothing in Part IV changes the type system (Section 11), the operator set (Section 13), the semantics (Section 15), or the second layer of the learning algorithm (Section 16.3). It changes only how the pool of candidate predicates handed to that second layer is built and stored.

21. The value bank

21.1 Why the tree stops being the right picture

In the Concept Tree Learner each node is a predicate, a path from the root is a conjunction of predicates, and the string threaded along that path is progressively trimmed. The only argument a node ever receives is what its parent hands down. Under that restriction "tree of paths" is an honest description of the search state: every candidate is a root-to-leaf path, and paths are independent.

That restriction is what CGL removes. Once `COUNT(INPUT, '0')` computed for one purpose can serve as the second argument of `TRIMHEAD` somewhere else, candidates stop being paths. Two consequences follow immediately.

The first is that the natural unit of search is no longer a candidate hypothesis but a **derived value**. What the search accumulates is a pool of values obtainable from the input, of every type, each available as an argument anywhere a value of that type is expected. Whether a particular value was originally built "for" a comparison or "for" a trim is not a property of the value and is not recorded.

The second is that deduplication changes status. In Section 4.3 observational equivalence is presented as an optimisation: it makes an otherwise intractable enumeration finish. Here it is structural. The argument pool of a node is the whole bank, so the branching factor of the search is `|Ops| × |Bank|^a` where `a` is the maximum arity. If the bank stores expressions, `|Bank|`

grows as fast as the expression space and the search does not merely slow down, it fails to make progress: almost every tuple it forms is a re-derivation of a value it already has. Keying the bank by behaviour is therefore a precondition for cross-branch arguments being usable at all, not a speed-up applied afterwards.

21.2 Definition

Fix an ordered list of ingested strings `S = (s₁, …, s_m)`. As in Section 15.3 these are the training strings with labels excluded; the bank never sees a label, so Layer 1 remains label-free and the leakage guard of Section 10.4 stays structural.

A **value bank** over `S` is a tuple `B = (C, τ, β, P, μ)`:

Component	Type	Meaning
`C`	finite set	class identifiers, opaque and stable
`τ`	`C → Type`	the type of every expression in the class
`β`	`C → Value^m`	the class's behaviour vector
`P`	`C → set of productions`	how the class can be built
`μ`	`C → ℕ`	the size of the smallest expression in it

A **production** is a pair `(op, ⟨c₁, …, c_k⟩)` with `k = arity(op)`, `τ(cᵢ) = op.paramType(i)` for every `i`, and `op.resultType = τ(c)` for the class `c` that holds it. Productions of arity zero are the constants of Section 13.2 and `INPUT`.

The **value of a production** at position `j` is obtained by applying the operator to the arguments' behaviours at that position, with the strict propagation of Section 12:

```
ev(op, ⟨c₁, …, c_k⟩)[j]
    = ⊥                                        if some β(cᵢ)[j] = ⊥
    = op.apply( β(c₁)[j], …, β(c_k)[j] )       otherwise
```

A class is therefore an equivalence class of expressions under observational equivalence on `S`, represented not by its members but by the ways of producing it. **The member expressions are never stored.** Recovering one is the job of Section 21.5.

21.3 Invariants

Checked on construction, and after every ingestion in Section 22.

- **B1 — Congruence.** For every class `c` and every production `π ∈ P(c)`, `ev(π) = β(c)`. Every recorded way of building the class agrees with the class's behaviour, coordinate by coordinate. This is the invariant that Section 22 exists to restore.
- **B2 — Quotient.** Distinct classes have distinct keys: `c ≠ c′ ⇒ (τ(c), β(c)) ≠ (τ(c′), β(c′))`. The bank is a quotient of the expression space, not a selection from it.
- **B3 — Inhabitation.** Every class has at least one production. Classes are created only by inserting a production, so an empty class cannot arise.

- **B4 — Non-degeneracy.** No class has $\beta(c) = (\bot, \ldots, \bot)$. An expression undefined on every training string cannot contribute to anything, since $\bot$ propagates, and admitting such classes floods the bank.
- **B5 — Minimality of μ.** $\mu(c) = \min$ over $\pi = (op, \langle c_1, \ldots, c_k \rangle) \in P(c)$ of $1 + \Sigma_i \mu(c_i)$.

Note what is *not* an invariant. The relation "class `c` has a production consuming class `d`" **may contain cycles**, and this is normal rather than a defect. If every training string happens to be a palindrome, `REVERSE(INPUT)` has the same behaviour as `INPUT`, so the class of `INPUT` acquires a production consuming itself. Likewise the class whose behaviour is constantly `true` acquires the production `AND(c, c)`. The structure is an e-graph, and e-graphs are cyclic in general.

What B5 guarantees instead is that the *extraction* sub-graph is acyclic. For the production attaining the minimum, $\mu(c) = 1 + \Sigma_i \mu(c_i) > \mu(c_i)$ for every i, because every class has $\mu \geq 1$. Choosing the μ-minimising production at each class therefore always descends strictly, and extraction terminates. Cyclic productions are recorded, are semantically correct, and are simply never selected.

The palindrome example is also a reminder of the caveat in Section 4.3: equivalence is relative to S. A class merges expressions that the training data cannot tell apart, and that is exactly the source of both the collapse in search cost and the risk of picking the wrong representative.

21.4 Batch construction

The enumeration order is unchanged from Section 16.2 — strictly increasing size, which is what makes the first consistent hypothesis provably smallest. What changes is the handling of a behaviour that has been seen before. Section 16.2 discards the candidate. Here the candidate's production is recorded on the existing class, and only the expansion is suppressed.

```
BUILD_BANK(S, Ops, maxSize, poolCap, pMax):
    C ← ∅ ; index ← empty map     # (Type, behaviour) → class
    byType ← empty map            # Type → list of classes

    for each op in Ops with arity 0:
        ADD(op, ⟨⟩, 1)

    for size = 2 to maxSize:
        for each op in Ops with arity k ≥ 1:
            for each tuple ⟨c₁, …, c_k⟩ such that
                    cᵢ ∈ byType[ op.paramType(i) ]  and
                    1 + Σᵢ μ(cᵢ) = size:
                ADD(op, ⟨c₁, …, c_k⟩, size)
        if |C| > poolCap:
            PRUNE(C)              # drop the largest-μ first
    return byType[Bool]

ADD(op, args, size):
    b ← ev(op, args)
```

```
    if b = (⊥, …, ⊥):
        return NONE                    # B4
    c ← index[(op.resultType, b)]
    if c = NONE:
        c ← fresh identifier
        τ(c) ← op.resultType ; β(c) ← b ; μ(c) ← size ; P(c) ← ∅
        index[(op.resultType, b)] ← c
        byType[τ(c)].append(c)
    RECORD(c, (op, args))
    return c

RECORD(c, π):
    P(c) ← P(c) ∪ { π }
    if |P(c)| > pMax:
        keep the pMax productions of smallest 1 + Σᵢ μ(cᵢ),
        always retaining the one attaining μ(c);
        ties by a fixed deterministic operator order
```

Four points on why this is the right shape.

Cross-branch arguments are not a special case. The tuple loop draws each argument from `byType` — the whole bank of the required type — with no notion of where a class came from. `TRIMHEAD(INPUT, COUNT(INPUT, '0'))` is formed by the same rule as any other size-5 expression. There is no wiring step, no branch bookkeeping, and no danger of missing a combination.

The iteration is over classes, not expressions. This is the entire economy of the structure. Each class enters the tuple loop once no matter how many expressions it stands for.

Productions are recorded even when the class already exists. This is what distinguishes the bank from Section 16.2's `Seen` set, and it costs one set insertion. Three later features need it and none of them can be retrofitted: extraction of *all* minimal hypotheses rather than one (requirement F6, Section 19.4), the identifiability report of Section 19.4, and incremental refinement (Section 22). Suppressing the *expansion* is what keeps the cost down; suppressing the *record* would save almost nothing and lose all three.

Productions must be capped. Classes saturate — Section 21.6 — but productions do not: a class can be the target of arbitrarily many tuples, and at large `maxSize` the memory of the bank is dominated by productions rather than by classes. `pMax` (default 8) bounds this. The retention rule keeps the μ-attaining production, so B5 survives the cap and extraction is unaffected; what is lost is some of the alternatives reported for identifiability, which should be stated in the report as a truncation rather than silently dropped.

21.5 Extraction

A class is turned back into a hypothesis by choosing, at each class reached, the production attaining μ, and memoising so that a class reached twice yields the same node twice-referenced:

```
EXTRACT(B, c):
    memo ← empty map
```

```
    n ← NODE(c, memo)
    return the concept graph with output n,
           unreachable nodes stripped

NODE(c, memo):
    if c ∈ memo:
        return memo[c]
    (op, ⟨c₁, …, c_k⟩) ← the production of P(c) minimising
                         1 + Σᵢ μ(cᵢ), ties broken by a
                         fixed deterministic order
    n ← new node with op(n) = op and
                      args(n) = [ NODE(cᵢ, memo) for i = 1..k ]
    memo[c] ← n
    return n
```

The memo is the reason the output is a DAG rather than a tree, and hence the reason a hypothesis satisfies Section 14 as written. Sharing is not detected after the fact by a common-subexpression pass; it is what the bank was storing all along.

One discrepancy must be recorded honestly, because it is a real gap between the search order and the objective. μ is a **tree** size: the recurrence `1 + Σᵢ μ(cᵢ)` charges for a shared class once per use. Section 14.3 defines `size(G)` as a **DAG** size, charging once in total. So `size(EXTRACT(B, c)) ≤ μ(c)`, with strict inequality exactly when extraction reuses a class. The enumeration is therefore ordered by an upper bound on the quantity the objective minimises.

This is not repaired by changing the extraction rule. Choosing productions so as to minimise DAG size — where the benefit of a choice depends on what other choices reuse it — is the standard e-graph extraction problem under a sharing-aware cost, and it is NP-hard. The specification therefore takes the practical position:

1. Search and enumerate by μ, which is cheap, monotone, and preserves the smallest-first guarantee *with respect to tree size*.
2. Report `size(G)` from Section 14.3 as the size of the delivered hypothesis, and use it in `COST` (Section 16.4).
3. State in the report that minimality is claimed for tree size. This is the same class of substitution as the one flagged for priority-weighted search: the guarantee that is proved and the guarantee that is claimed must be the same sentence.

21.6 Saturation, and what does not saturate

The bound of Section 4.3 carries over unchanged and is the reason the structure is finite. With `m` ingested strings, a `Bool`-valued class is determined by an element of `{true, false, ⊥}^m`, so there are at most `3^m` of them. `Char` classes are bounded by `(|Σ| + 1)^m`. `Int` and `Str` classes have no useful a-priori bound, but in practice are limited by the distinct value tuples the operators actually produce from `m` short strings, which is small.

Three quantities behave differently and should be measured separately in the cost report of Section 19.3:

Quantity	Growth	Bounded by
classes `|C|`	saturates	`3^m` for `Bool`; empirical for `Int`, `Str`
productions `Σ|P(c)|`	grows with `maxSize`	`pMax · |C|`
tuples examined	`|Ops| · |C|^a` per level	the dominant cost

The third is the one that decides whether a run finishes. It depends on `|C|`, which saturates, and not on the number of expressions, which does not — which is the whole argument for the structure.

21.7 Behaviour equivalence is finer than coverage equivalence

The Concept Tree Learner deduplicates its leaf paths by **coverage**: the vector of which positive examples a path accepts. Measured across the twenty-nine test cases at depth 4, that collapses 1,267,369 leaf paths to 367 distinct classes, a factor of 3,453, and reduces the input to set cover from a mean of 2,458 paths per case to 12.7.

That equivalence is sound there for a specific reason: the objective of the tree learner sees a path only through its coverage of the positives and its depth, so two paths with equal coverage are interchangeable *as rules*. It is **not** sound here, and the difference is worth stating explicitly because the two look similar and the weaker one is cheaper.

Keyed by	Sound for	Unsound for
coverage of positives (CTL)	selecting among finished rules by the Layer-2 objective	composition
behaviour on all of `S`, positives and negatives alike, including `⊥` (CGL)	composition, hence use as an argument	—

Two `Bool` expressions that agree on every positive but differ on a negative are interchangeable as rules and *not* interchangeable as arguments: `NOT` of the first and `NOT` of the second differ, and either may be the argument that makes some larger expression consistent. Merging them would make the bank unsound as a source of arguments, which is precisely the failure mode recorded in the project history for optimistic subtree sharing in the tree learner.

The practical consequence is a prediction that the evaluation should check rather than assume: **the collapse factor here will be smaller than 3,453**, because the equivalence is finer — it distinguishes on negatives and on `⊥` as well as on positives. If the measured factor comes out comparable, that is evidence the negatives are not discriminating, which is itself worth reporting.

21.8 Configuration additions

Extending the table of Section 18:

Parameter	Default	Meaning
`pMax`	8	Productions retained per class (Section 21.4)

Parameter	Default	Meaning
`ingestMode`	`batch`	`batch` or `incremental`; see Section 22
`bankReport`	`false`	Emit per-level class, production and tuple counts

22. Incremental ingestion and partition refinement

Status: optional extension. `BUILD_BANK` is normative for stage 3. This section specifies what is required when the training strings are not all known before the bank is built. It is written now, and not later, because two of its requirements are free if they are respected from the start and expensive to retrofit — Section 22.7 states them.

22.1 The conflict

Deduplication and incrementality pull against each other. Two expressions that agree on $s_1, \ldots, s_k$ are merged; the next string may distinguish them. The merge was not wrong when it was made and is wrong afterwards, so what is needed is not a way to avoid merging but a way to undo one soundly.

Undoing is impossible if a class is stored as a representative expression, because the discarded members are gone. It is straightforward if a class is stored as its productions, because a production is a *schema*: $(op, \langle c_1, \ldots, c_k \rangle)$ denotes the whole set $\{ op(e_1, \ldots, e_k) : e_i \in c_i \}$. Splitting the argument classes partitions that set without anyone ever having enumerated it. This is why B3 requires productions rather than members, and it is the single design decision that makes the rest of this section possible.

22.2 What ingestion must do

Appending a string s extends every behaviour vector by one coordinate. A class whose productions all take the same value at the new coordinate simply grows a coordinate. A class whose productions disagree there was never one class under the finer observation, and must be replaced by one class per value.

Splitting an argument class changes the value of every production consuming it, which may split the consumer, and so on. Restoring B1 under the extended observation is therefore a congruence closure, computed by the standard worklist:

```
INGEST(B, s):
    children ← map, initially children(c) = {c} for every class c
    W ← all classes, ordered by increasing μ

    while W ≠ ∅:
        c ← W.pop()

        # re-point productions through split arguments
        P′ ← ∅
        for each (op, ⟨c₁, …, c_k⟩) ∈ P(c):
            for each ⟨d₁, …, d_k⟩ ∈
```

```
                children(c₁) × … × children(c_k):
            P′ ← P′ ∪ { (op, ⟨d₁, …, d_k⟩) }

    groups ← partition P′ by ev_s(·)    # new coordinate

    if |groups| = 1:
        β(c) ← β(c) ++ [ the single value ]
        P(c) ← P′
    else:
        for each group g ∈ groups:
            cg ← fresh identifier
            τ(cg) ← τ(c)
            β(cg) ← β(c) ++ [ value of g ]
            P(cg) ← g
            μ(cg) ← recomputed by B5
        children(c) ← { cg : g ∈ groups }
        remove c from C ; add the fresh classes
        W.push( every class consuming c )

drop any class violating B4
recompute μ to a fixed point ; return B
```

The re-pointing step is the part an implementation is most likely to get wrong. A production is *not* redirected to whichever child "the original expression" fell into — there is no original expression. It is expanded into the product of its arguments' children, and each expansion is evaluated on its own. Expansions that turn out to have equal values simply land in the same group.

22.3 Invariants

- **R1 — Congruence restored.** On return, B1 holds over `m + 1` coordinates.
- **R2 — Refinement only.** The partition after ingestion refines the partition before it. Two classes distinguished by the first `m` strings are still distinguished after the next one, so **classes never merge**. Nothing in the algorithm needs a merge case, and an implementation that appears to need one has a bug elsewhere.
- **R3 — Identity is stable except on split.** A class that does not split keeps its identifier. A class that splits ceases to exist, and every external reference to it — an extracted hypothesis, an entry in the concept store — is invalidated and must be revisited. This is the hook the store's dependency tracking attaches to.
- **R4 — μ never decreases.** Each child inherits a subset of its parent's productions, so its minimum can only rise. A hypothesis is therefore never made smaller by seeing more data, and a lower bound established before ingestion remains a lower bound after it.

Termination. Each pass either settles a class or strictly refines the partition. The partition is bounded below by the finest partition of the expressions present, which is finite because behaviours over `m + 1` coordinates are finite, so only finitely many splits can occur. Cyclic productions — Section 21.3 — are handled by the worklist rather than by the ordering, which is why the ordering by μ is an efficiency heuristic and not a correctness requirement.

Cost. A single split costs the re-evaluation of the productions of the split class and enqueues its consumers, so its cost is proportional to `pMax` times the number of classes consuming it. The amortised statement is the useful one: because the partition only ever refines, the total number of splits across *all* ingestions is bounded by `|C_final| − |C_initial|`. Incremental construction therefore does not repeat the work of batch construction once per example; it pays, in total, for the splits that distinguish the classes the batch build would have created directly.

22.4 Identifiability comes out of this for free

A `Bool` class that has survived every ingestion while holding two or more productions of equal μ is exactly a set of distinct minimal expressions that the data cannot tell apart. That is the set `C` of Section 16.4 and the count reported in Section 19.4, available by inspection rather than by a separate search.

The disambiguating example of Section 19.4 has an equally direct reading: it is a string `s` on which two productions of one class take different values. Confirming it is one call to `INGEST` — the class splits, and the report can name which hypotheses went which way. Under batch construction the same question requires rebuilding the bank from scratch with the candidate string appended, which is the reason an active-learning loop wants this section and a fixed-training-set run does not.

22.5 When refinement earns its keep

Situation	Why batch is not enough
Active learning / membership queries (Sections 2.7, 19.4)	Each query appends a string chosen *because* the bank cannot resolve it; rebuilding per query is quadratic in the number of queries
Concept store (stage 4)	Entries outlive the training set they were learned from and must remain valid, or be explicitly invalidated, when new data arrives
Optimistic sharing in the tree learner	Sharing a subtree between two nodes is sound only if they produce the same residual for every word, which is unknown during incremental construction; merge-then-refine is the sound form of it
Streaming or oversized `S`	Ingest a prefix, prune, continue, rather than holding every behaviour vector at full width

Where none of these applies, refinement is pure overhead: with `S` fixed and known, `BUILD_BANK` computes the fully refined partition directly, in one pass, and no split ever occurs.

22.6 Relation to Section 16.2

Section 16.2's `ENUMERATE` is the special case of `BUILD_BANK` in which productions are discarded on insertion and `pMax = 1`. Section 22 is the general case of ingestion in which `INGEST` is called once per string as it arrives; calling it `m` times before any query, on a bank

built from no strings at all, reproduces `BUILD_BANK` — more slowly, and with no other difference in the result.

22.7 What stage 3 must do now

Two requirements, both free in a batch implementation and both expensive later:

1. **Store productions, not members.** `RECORD` must run on every insertion, including the ones whose behaviour already exists. Under batch construction the recorded productions are used only for identifiability reporting, so the temptation is to skip them under a flag. Skipping them makes Section 22 a rewrite rather than an addition.
2. **Class identifiers must be opaque and stable.** No part of the system may index a class by its position in a list, and no part may assume the set of classes only grows. `children` in Section 22.2 replaces one identifier with several; every consumer of the bank must be able to survive that. A dictionary keyed by identifier is fine; an array index into `byType` is not.

Neither costs measurable time. Both are load-bearing for stages 4 and 5.

PART III — APPENDICES

Appendix A. Worked example: Test Case 1024

This appendix runs the algorithm of Section 16 by hand on the test case from the original document. It illustrates every major idea in Part I, and it ends with a result that is more instructive than expected.

A.1 The data

```
+ 1001
+ 00001
- 11100
+ 10
- 111
```

Five examples, three positive. Stated concept: *the longest run of consecutive 1s is at most 2.* Target behaviour vector, in file order:

```
target = ( T, T, F, T, F )
```

A.2 Some behaviour vectors

Layer 1 evaluates candidate expressions on the five strings, ignoring labels entirely. A few useful `Int`-valued expressions:

Expression	1001	00001	11100	10	111
`LEN(INPUT)`	4	5	5	2	3

Expression	1001	00001	11100	10	111
`COUNT(INPUT,'1')`	2	1	3	1	3
`MAXRUN(INPUT,'1')`	1	1	3	1	3
`DEC(INPUT)`	9	1	28	2	7

Note immediately that `LEN` is useless on its own: the value 5 occurs for both a positive (`00001`) and a negative (`11100`), so no predicate over `LEN` alone can separate them. The search discovers this automatically — no `Bool` expression built from `LEN` alone will ever have the target behaviour — but it is worth seeing why.

A.3 Enumeration by size

Size 1. `INPUT`; integer constants `0`…`8`; characters `'0'`, `'1'`; string constants of length ≤ 3 excluding any training string; `TRUE`, `FALSE`. The two boolean constants have behaviours `(T,T,T,T,T)` and `(F,F,F,F,F)`; neither is the target.

Size 2. `LEN(INPUT)`, `REVERSE(INPUT)`, `HEAD(INPUT)`, `LAST(INPUT)`, and so on. No `Bool` expressions yet except negations of the constants, whose behaviours are already in `Seen` and are discarded.

Size 3. The first interesting booleans appear — an operator plus `INPUT` plus a constant:

Expression	Behaviour	Target?
`STARTSWITH(INPUT, "1")`	T F T T T	no
`ENDSWITH(INPUT, "1")`	T T F F T	no
`CONTAINS(INPUT, "0")`	T T T T F	no
`CONTAINS(INPUT, "11")`	F F T F T	no
`CONTAINS(INPUT, "111")`	F F T F T	**duplicate**

The last row is the first illustration of observational equivalence: `CONTAINS(INPUT,"11")` and `CONTAINS(INPUT,"111")` produce identical values on all five training strings, so the second is discarded as a duplicate. They are of course different programs — they disagree on the string `11` — but the training data cannot tell them apart. Hold onto this; it is the whole point of the appendix.

No size-3 expression has the target behaviour.

Size 4. Negations of the size-3 booleans, and comparisons involving `LEN`:

Expression	Behaviour	Target?
`NOT(CONTAINS(INPUT, "0"))`	F F F F T	no
`EQ(LEN(INPUT), 5)`	F T T F F	no
`LE(LEN(INPUT), 4)`	T F F T T	no
`CHREQ(HEAD(INPUT), '1')`	T F T T T	no
`NOT(CONTAINS(INPUT, "11"))`	**T T F T F**	**yes**

A consistent hypothesis at size 4. Because enumeration proceeds in strict size order, this is provably a *smallest* consistent hypothesis, and the algorithm returns at this level without ever building the size-5 expressions.

A.4 The result is not what the data's author intended

The learner returns:

```
NOT(CONTAINS(INPUT, "11"))
```

which reads *"the string does not contain two consecutive 1s"* — that is, **at most one** consecutive 1. The intended concept was **at most two**.

Nothing has gone wrong with the algorithm. Occam's razor did exactly what it was asked to do, and the returned hypothesis classifies all five training examples correctly. The problem is in the data: **no positive example contains 11**. The boundary case that would distinguish "at most 1" from "at most 2" is simply absent, exactly as Section 2.7 warned.

This is the single most valuable thing this worked example demonstrates. A learner that reported only `NOT(CONTAINS(INPUT,"11"))` with "100% training accuracy" would be actively misleading.

A.5 What requirement F6 produces instead

Section 16.4 returns the *set* of minimal consistent hypotheses, not one element. At size 4 that set is:

```
C = {  NOT(CONTAINS(INPUT, "11")),      # at most 1 consecutive 1
       NOT(CONTAINS(INPUT, "111"))  }   # at most 2 consecutive 1s
```

(The second was discarded from the bank as observationally equivalent to `CONTAINS(INPUT,"11")` at size 3, so an implementation must retain discarded duplicates in a side list precisely so that this report can be produced. That is a concrete design consequence of requirement F6 and is easy to overlook.)

The identifiability report of Section 19.4 therefore reads roughly:

> Two hypotheses of minimal size 4 are consistent with these 5 examples. They disagree on the string `11`. Add a labelled example for `11` to distinguish them. Sample-

complexity estimate for this hypothesis space suggests at least 40 examples; this result is **not identified** by the data.

A.6 Adding the disambiguating example

Adding `+ 11`, plus two more examples for margin, gives the extended test case of Section 10.2:

```
+ 1001     + 00001    - 11100    + 10
- 111      + 11       - 01110    + 110011
```

Now:

- `NOT(CONTAINS(INPUT,"11"))` on `+ 11` predicts `false` — an error. Eliminated.
- `NOT(CONTAINS(INPUT,"111"))` on `+ 11` → `T` ✓, on `- 01110` → `F` ✓, on `+ 110011` → `T` ✓. Still consistent.

The learner now returns a single minimal hypothesis, and the report says the data identify it. Equivalently, at size 5 the search would also find `LE(MAXRUN(INPUT,'1'), 2)`, which is the same concept in the form the document's author had in mind. Both are correct; the size-4 form is returned because it is smaller.

A.7 What this example demonstrates

Concept	Where it appeared
Behaviour vector (§3.3, §15.3)	The tables in A.2 and A.3
Observational equivalence (§4.3)	`CONTAINS "11"` vs `CONTAINS "111"` at size 3
Size-ordered enumeration guarantees minimality (§4.2)	Returning at size 4
Occam's razor (§2.6)	Preferring size 4 over size 5
Identifiability failure (§2.7)	Two minimal hypotheses, both consistent
Active learning / disambiguating query (§2.7)	Requesting a label for `11`
Expressiveness (§2.2, §13.4)	None of this works without `CONTAINS` on strings or `MAXRUN`; the original operator set had neither

Appendix B. Change log against the original document

Each entry gives the change, the reason, and the section of this document that contains the replacement.

B.1 Blocking corrections

#	Original	Problem	Resolution	§
1	"the graph is simultaneously	No definition of what a learned	Denotational semantics	15

#	Original	Problem	Resolution	§
	traversed and created to match the input string”	graph computes on an unseen string; the output was not a well-defined function	`⟦G⟧ : Σ* → Bool`, single `Bool`-typed output node	
2	“any successful operation is appended”, success = “no exceptions”	A type-safety test, not a learning criterion; exerts no pressure toward separating classes; admits the rote learner as perfect	Explicit MDL objective `COST(H)`	2.6, 16.4
3	Construction shown only for positive examples	Negatives never constrain the hypothesis; the always-`true` graph is never penalised	Batch scoring over all examples; Layer 2 uses negatives to terminate each rule	16.3
4	“it is important that the input is sorted in parity”	Unjustified; presenting all positives first drives over-generalisation	Removed. All candidates are scored against the whole example set, so order is irrelevant	16.2
5	“Root always takes the full input line as input”	Label leakage: `STARTSWITH(INPUT,"+")` would be a perfect classifier	Parser separates `(label, string)`; the evaluator’s signature takes a string	10.4
6	Operator set had no `AND/OR/NOT`, no inequality, no `SUB/MOD`, and `Has` took a character not a string	**None of the four concepts named in the original’s own introduction were expressible.** Nor was Test Case 1024’s own	Boolean connectives, six comparisons, `SUB/DIV/MOD`, `MAXRUN`, `COUNTSUB`, string-valued `CONTAINS`	13

#	Original	Problem	Resolution	§
		concept		
7	No inference mode; output was “the graph”	A learner with no `classify` cannot be tested, and its search cannot score candidates	`CLASSIFY(G, s)` as a public entry point	15.2
8	No held-out evaluation, no baselines	Training-set accuracy of a consistent hypothesis is always 100% and measures nothing	Full evaluation protocol with generated test sets and three baselines	19

B.2 Specification defects

#	Original	Problem	Resolution
9	“Check nodes do not have results” (prose) vs. SW has an Output Value of true/false (table)	Booleans in two different slots	`Bool` is an ordinary result type
10	“Alter nodes do not produce results” vs. TRH Output Value = “removed part”	Direct contradiction	Uniform: one result per operator
11	“Operator nodes do not produce results” vs. ADD Output Value = `m+n`	Direct contradiction	As above
12	`BEQ` declared a Check node but takes a string argument, contradicting “one character and one numeric argument”	Type rule violated by its own table	`STREQ : Str × Str → Bool`
13	`DEQ` declared a Check node taking a Double, contradicting “Input: a binary string”	Type rule violated	`EQ : Int × Int → Bool`
14	`End` classified as an Alter node	It neither alters nor returns a string; and multiple `End` nodes	Removed; termination is by output type

#	Original	Problem	Resolution
		had no combination rule	
15	`DEC` returns “Double”	Loses precision above 2^{53}, so distinct strings compare equal with no error raised	Arbitrary-precision `Int`, with $\perp$ above `MAX_DEC_LEN`
16	`HD`/`TL` return characters, but operator nodes were specified over “two numerics, two characters, or two binary strings” while `ADD` was over doubles	`Char` was not a first-class type; `ADD` on chars undefined	Four explicit types, one signature per operator
17	Every node carries a `Flag` = “no exceptions”	Exceptions as control flow in a search inner loop; “threw” is not a composable value	Total operators with $\perp$; `Flag` is $v \neq \perp$ and is not stored
18	Separate “Input” and “Argument”, but arguments could be node outputs	The distinction dissolves; the worked example used it inconsistently	Single ordered typed parameter list; constants are 0-arity operators
19	Input file format given only by example	Comments, blank lines, empty strings, malformed lines, duplicate and conflicting examples all undefined	EBNF grammar plus an explicit edge-case table
20	Alphabet fixed to `{0,1}`	Needless restriction	`alphabet` declared in the file header

B.3 Editorial

#	Location	Correction
21	Test Case 1024 description	“at most consecutive 1s” is missing its number; should read “at most **2** consecutive 1s”, per the following sentence
22	Test Case 1024 data	Contains no positive example with `11`; the concept is not

#	Location	Correction
		identifiable from it (Appendix A)
23	Algorithm section	The ASCII graph does not render; branch lines are misaligned. Redraw as a figure
24	Algorithm section	`DEQ(1)` is drawn with a constant argument but described in prose as taking two node outputs. Correct the figure to match
25	Throughout	“preceeded” → “preceded”; “cnverted” → “converted”; “succesful” → “successful”; “randomicity” → “randomness”

B.4 What was kept

It is worth being explicit about what survives, because the redesign is narrower than the change list suggests. The core idea of the original — that a concept should be represented as a typed graph of small operations, assembled from a fixed vocabulary, and searched for rather than hand-written — is retained unchanged, and is a good idea. What changed is that the graph now has a defined meaning, the vocabulary can express the intended targets, and the search has something to optimise.

Appendix C. Suggested implementation milestones

Ordered so that each milestone is independently testable and produces something demonstrable.

#	Milestone	Deliverable	Depends on
1	Types, values, ⊥, operator interface	Unit tests: every operator total and pure on a fixed input battery	—
2	Operator registry and the full set of Section 13	Registry lists every operator with its signature	1
3	Concept graph, invariants I1–I4, type checker	Property tests: malformed graphs rejected	1
4	Evaluator with	Hand-written	3

#	Milestone	Deliverable	Depends on
	memoisation; `CLASSIFY`	hypotheses classify hand-written strings correctly	
5	Formula renderer	Round-trip: render then re-parse a hypothesis	3
6	Parser with the grammar and edge cases of Section 10	Every row of the table in 10.3 covered by a test	—
7	Layer 1 enumeration with dedup	Reproduces the tables in Appendix A.2 and A.3 exactly	4, 6
8	Objective function	Rote-learner regression test from Section 13.10 fails to win	4
9	Top-level learner, single-predicate case	Solves Test Case 1024 and reports both minimal hypotheses	7, 8
10	Layer 2 sequential covering	Solves a concept requiring a conjunction of two predicates	9
11	Evaluation harness, generated test sets, metrics	Full report for one test case	9
12	RPNI baseline	Comparison table for the regular concepts	11
13	Evolutionary strategy	Same interface; comparison against enumeration	8, 11
14	*(stretch)* `DFA` operator	A concept mixing automaton acceptance with arithmetic	12

Milestones 1–9 constitute a complete working system for the concepts in the original document. Everything after 9 is comparison and extension.

Appendix D. References

Concept learning and learning theory

- T. Mitchell, *Machine Learning*, McGraw-Hill, 1997. Chapters 2 (concept learning, hypothesis spaces, inductive bias), 3 (decision trees), 6 (Bayesian view of Occam's razor), 7 (PAC learning, VC dimension), 10 (rule learning and FOIL).
- A. Blumer, A. Ehrenfeucht, D. Haussler, M. Warmuth, "Occam's Razor", *Information Processing Letters* 24(6), 1987.
- J. Rissanen, "Modeling by shortest data description", *Automatica* 14(5), 1978. The origin of MDL.
- P. Grünwald, *The Minimum Description Length Principle*, MIT Press, 2007.

Rule learning

- P. Clark, T. Niblett, "The CN2 Induction Algorithm", *Machine Learning* 3(4), 1989.
- W. Cohen, "Fast Effective Rule Induction" (RIPPER), *ICML*, 1995.
- J. R. Quinlan, "Learning Logical Definitions from Relations" (FOIL), *Machine Learning* 5(3), 1990.

Program synthesis and genetic programming

- R. Alur et al., "Syntax-Guided Synthesis", *FMCAD*, 2013. The SyGuS framework and the enumerative solver architecture of Section 16.2.
- A. Udupa et al., "TRANSIT: Specifying Protocols with Concolic Snippets", *PLDI*, 2013. Bottom-up enumeration with observational equivalence.
- S. Gulwani, "Automating String Processing in Spreadsheets Using Input-Output Examples" (FlashFill), *POPL*, 2011.
- S. Gulwani, O. Polozov, R. Singh, "Program Synthesis", *Foundations and Trends in Programming Languages* 4(1–2), 2017. A good survey.
- J. Koza, *Genetic Programming*, MIT Press, 1992.
- D. Montana, "Strongly Typed Genetic Programming", *Evolutionary Computation* 3(2), 1995.
- J. Miller, P. Thomson, "Cartesian Genetic Programming", *EuroGP*, 2000. The representation closest to the original document's.

Grammatical inference

- D. Angluin, "Learning Regular Sets from Queries and Counterexamples" (L*), *Information and Computation* 75(2), 1987.
- J. Oncina, P. García, "Inferring Regular Languages in Polynomial Updated Time" (RPNI), 1992.
- K. Lang, B. Pearlmutter, R. Price, "Results of the Abbadingo One DFA Learning Competition and a New Evidence-Driven State Merging Algorithm" (EDSM), *ICGI*, 1998.
- C. de la Higuera, *Grammatical Inference: Learning Automata and Grammars*, Cambridge University Press, 2010. The standard textbook.

Semantics

- G. Winskel, *The Formal Semantics of Programming Languages*, MIT Press, 1993.Denotational semantics and the treatment of ⊥, for Section 15.